

Paradigmi di Programmazione - A.A. 2023-24

Esame Scritto del 7/11/2024

CRITERI DI VALUTAZIONE:

La prova è superata se si ottengono almeno 12 punti negli esercizi 1,2,3 e almeno 18 punti complessivamente.

Esercizio 1 [Punti 4]

Applicare la β -riduzione alla seguente λ -espressione usando la strategia **call-by-name** fino a raggiungere una espressione non ulteriormente riducibile o ad accorgersi che la derivazione è infinita:

$$(\lambda x.x)((\lambda x.\lambda y.xy)y)$$

Nella soluzione, mostrare tutti i passi di riduzione calcolati, sottolineando ad ogni passo la porzione di espressione a cui si applica la β -riduzione (redex) ed evidenziando le eventuali α -conversioni.

Esercizio 2 [Punti 4]

Indicare il tipo della seguente funzione OCaml, mostrando i passi fatti per inferirlo:

```
let test x y =  
  match y with  
  | (0,_,a) -> a=0  
  | (_,_,a) -> if x then a=0 else x;;
```

Esercizio 3 [Punti 7]

Definire in OCaml tre funzioni `coppie_da_lista`, `lista_tripla` e `espandi` con tipi

```
coppie_da_lista : 'a list -> 'a * 'a list  
lista_tripla : 'a * 'b * 'c list -> 'a list * 'b list * 'c list  
espandi : 'a * int list -> 'a list list
```

La funzione `coppie_da_lista` prende una lista $[e_1; e_2; e_3; \dots; e_k]$ di tipo `'a list` e restituisce una lista contenente tutte le possibili coppie che si possono formare prendendo due elementi qualunque della lista, come nel seguente esempio:

```
coppie_da_lista [1;2;3;4] = [(1, 2); (1, 3); (1, 4); (2, 3); (2, 4); (3, 4)]
```

La funzione `lista_tripla` prende una lista di triple $[(a_1, b_1, c_1); (a_2, b_2, c_2); \dots; (a_k, b_k, c_k)]$ di tipo `'a * 'b * 'c list` e restituisce una tripla di liste contenenti rispettivamente tutti i primi, tutti i secondi e tutti i terzi elementi delle singole liste, come nel seguente esempio:

```
lista_tripla [(1,5,4);(2,6,3);(3,7,2)] = ([1; 2; 3], [5; 6; 7], [4; 3; 2])
```

La funzione `espandi` prende una lista di coppie $[(e_1, n_1), (e_2, n_2), \dots, (e_k, n_k)]$ di tipo `('a * int) list` e restituisce una lista di tipo `'a list list` in cui ogni coppia (e_i, n_i) è sostituita da una lista con n_i copie dell'elemento e_i . Ad esempio:

```
espandi [( 'a', 4); ( 'b', 3); ( 'c', 5)] =  
  [[ 'a'; 'a'; 'a'; 'a']; [ 'b'; 'b'; 'b']; [ 'c'; 'c'; 'c'; 'c'; 'c']]
```

Esercizio 4 [Punti 15]

Si estenda il linguaggio MiniCaml visto a lezione con un nuovo tipo di dato che descrive valori definiti come coppie di valori più semplici, con le seguenti operazioni:

- `Pair(e1,e2)` definisce la coppia formata dai valori delle espressioni `e1` ed `e2`;
- `Frist(e)` fornisce il primo elemento della coppia descritta dall'espressione `e`
- `Second(e)` fornisce il secondo elemento della coppia descritta dall'espressione `e`

Ad esempio (in una sintassi concreta simile a OCaml):

```
Pair(3+2,false)           (* definisce la coppia (5,false) *)
First(Pair(3+2,false))    (* corrisponde a 5 *)
Second(Pair(3+2,false))   (* corrisponde a false *)
Pair(fun x -> x+1, fun x -> x-1) (* definisce una coppia contenente due funzioni *)
```

Inoltre, si introduca un costrutto `ComposeApply` che, data una coppia contenente due funzioni (`f1,f2`) e un parametro attuale `x`, restituisce il risultato dell'applicazione di `f2` al risultato dell'applicazione di `f1` a `x`.

- `ComposeApply(ep,earg)` dove `ep` valuta nella coppia (`f1,f2`) ed `earg` valuta nel valore `x`, calcola `f2(f1(x))`;

Un esempio articolato che mostra anche l'uso di `ComposeApply` (sempre in sintassi concreta simile a OCaml):

```
let f1 = fun x -> x+1 in
let f2 = fun x -> x*2 in
let p = (f1,f2) in
ComposeApply (p, 10);; (* il risultato è 22 *)
```

Si mostri come deve essere modificato l'interprete OCaml del linguaggio.